Міністерство освіти та науки України

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
Факультет інформатики та обчислювальної техніки

Кафедра інформатики та програмної інженерії

ЗВІТ

з дисципліни: «Програмування веб-застосувань»
Практична робота №14
Node.js: клієнт-серверна взаємодія, найпростіший сервер

Виконав:
Студент II курсу
групи ІП-56
Гончар І.В.

Перевірив:
Ковтунець О.В.

Київ 2026

1. Мета роботи:

Встановити Node.js і написати найпростіший локальний сервер (на вибір: вбудований модуль http чи Express) для свого застосунку: сервер має або роздавати статичні файли застосунку (index.html, CSS, JS), або віддавати JSON-дані вашого домену через власний ендпоінт замість зовнішнього публічного API з практикуму 9, або обидва варіанти одночасно. Клієнтський fetch має бути перемкнутий на локальний URL за тим самим патерном async/await + response.ok + try/catch, що й у практикумі 9 - лише з іншою адресою запиту.

2. Хід виконання роботи:

Кроки 1–4. Встановлення Node.js, ініціалізація проєкту та підготовка даних:

Перевірено встановлення середовища виконання Node.js (команди node -v та npm -v). У середовищі створено теку сервера та виконано npm init -y з подальшим встановленням пакету Express (npm install express) **(Рис. 1)**. Підготовлено локальний масив даних для предметної області «Каталог настільних ігор» (варіант 4), що містить записи із даними про ігри (id, title, minPlayers, maxPlayers, genre, img, owned, description, **duration**).

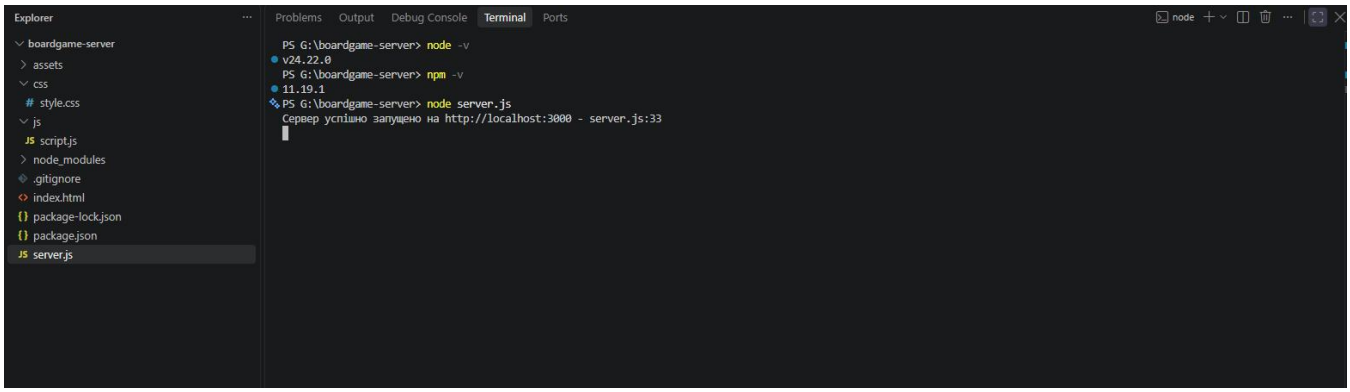


Рис. 1 — Термінал із перевіркою версій node/npm та запуском сервера

Кроки 5–7. Реалізація ендпоінтів API та роздача статики в Express:

У файлі server.js налаштовано Express-додаток. За допомогою middleware express.static налаштовано роздачу статичних файлів клієнтської частини (HTML, CSS, JS). Реалізовано GET-ендпоінт /api/games для отримання всього масиву ігор у форматі JSON **(Рис. 2)**, а також ендпоінт /api/games/:id для отримання конкретної гри за ідентифікатором або повернення коду 404.

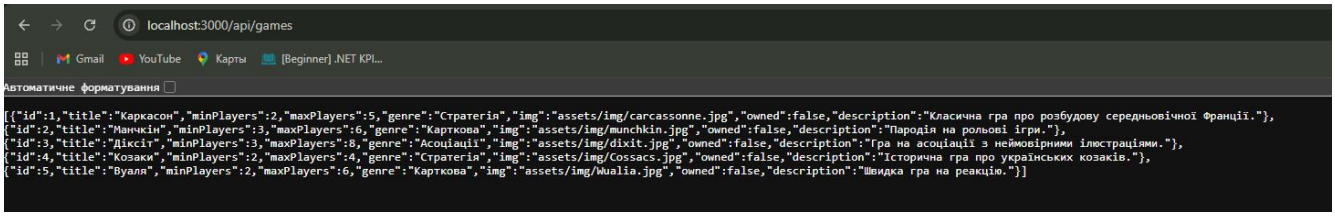


Рис. 2 — Відповідь ендпоінта /api/games у браузері

Кроки 8–10. Запуск сервера, перемикання клієнтського fetch та перевірка помилок:

Сервер запущено на порту 3000 (http://localhost:3000). У клієнтському JavaScript-коді адресу запитів fetch перемкнуто на локальні ендпоінти, що забезпечує повноцінну роботу та відображення інтерфейсу застосунку **(Рис. 3)**. Збережено надійну обробку помилок за патерном async/await із перевіркою response.ok та блоком try/catch, а успішне виконання локальних запитів підтверджено у DevTools **(Рис. 4)**. Проведено тестування запитів до неіснуючих ID, переконавшись, що сервер коректно повертає статус 404 та помилку **(Рис. 5)**.

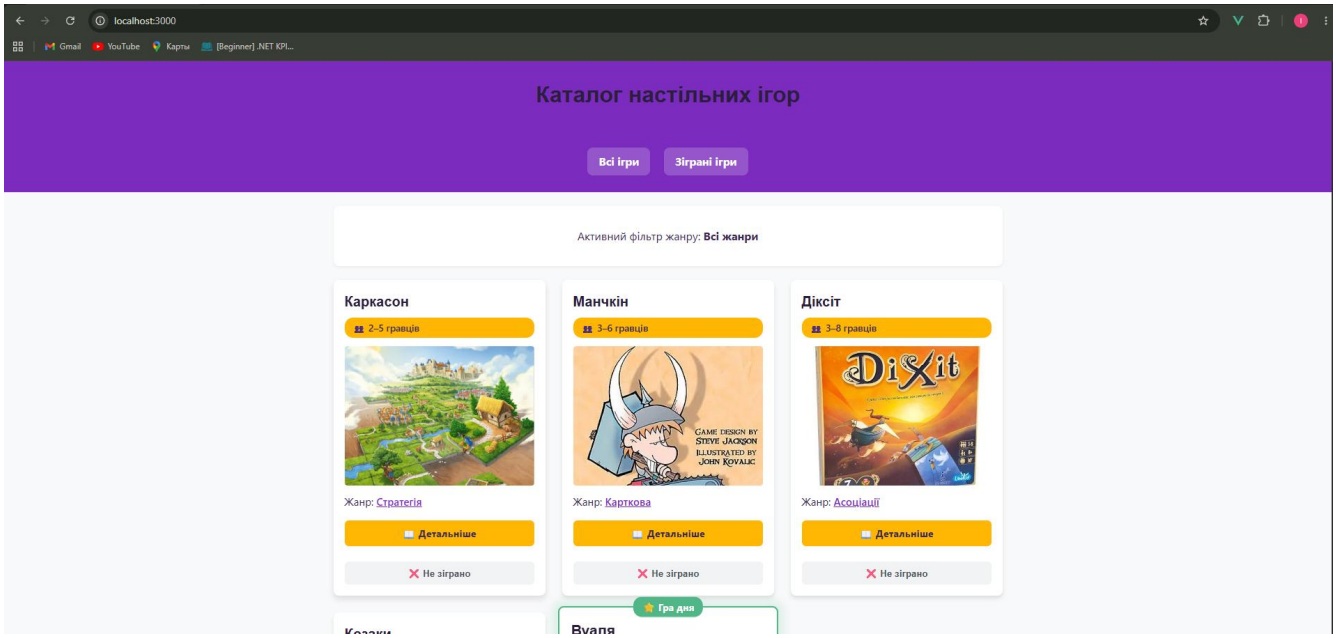


Рис. 3 — Робота застосунку в браузері через локальний сервер (localhost:3000)

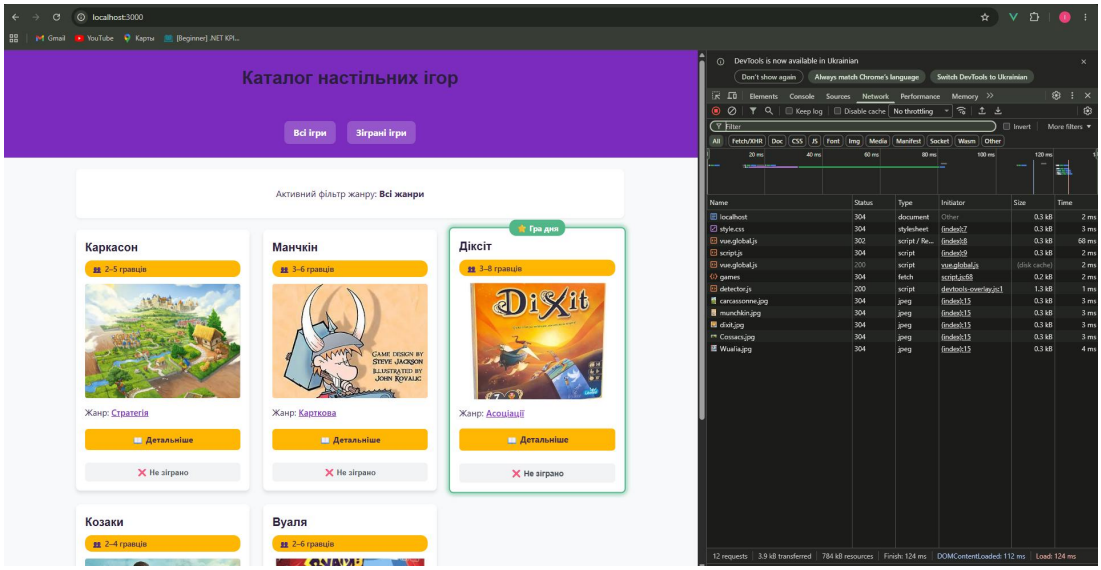


Рис. 4 — Вкладка Network у DevTools із успішними локальними запитами

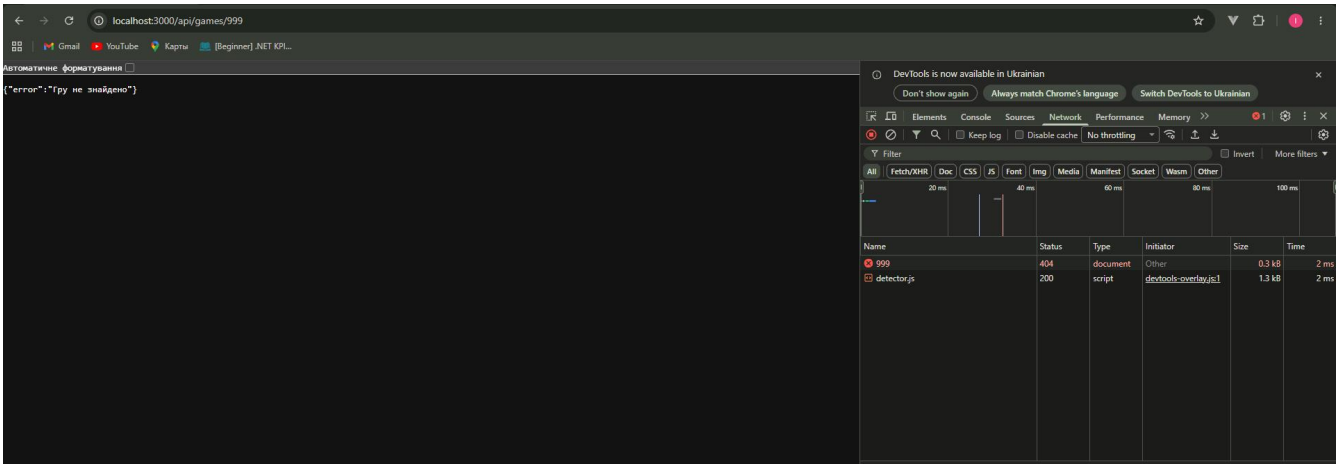


Рис. 5 — Перевірка обробки помилок: відповідь 404 для неіснуючого id (відповідь сервера та панель Network)

Крок 11. Версіонування та збереження:

Додано файл .gitignore для виключення теки node_modules із відстеження Git, код прокоментовано, а зміни зафіксовано у локальному репозиторії та відправлено на GitHub **(Рис. 6)**.

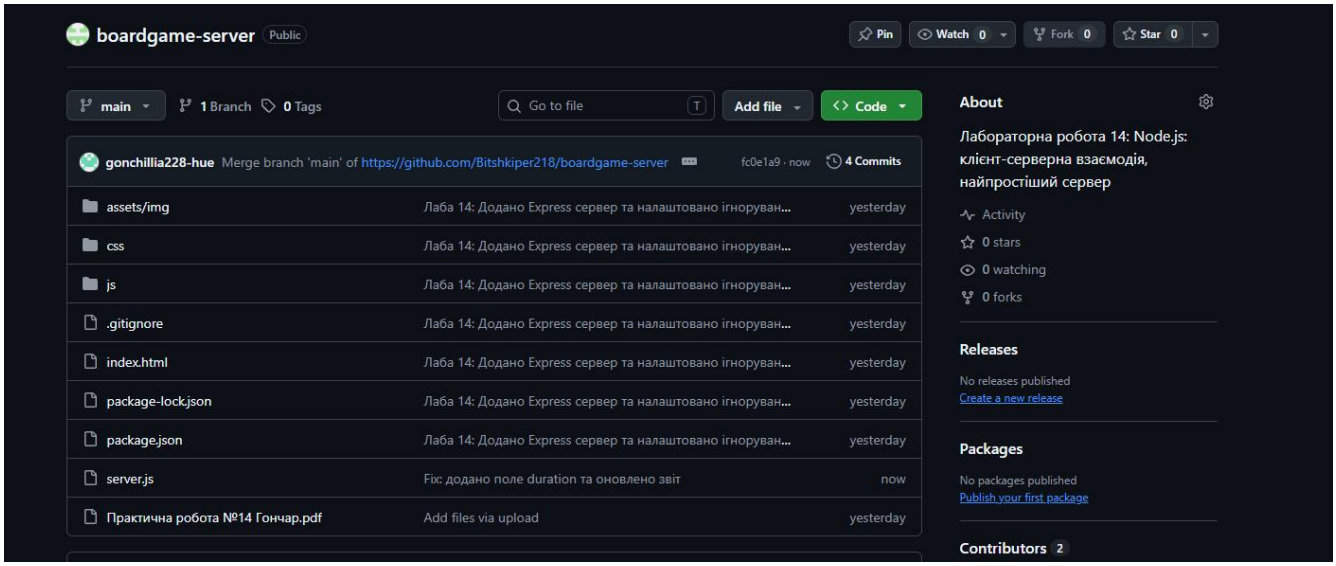


Рис. 6 — Фіксація змін та коміти у віддаленому репозиторії на GitHub

Посилання на репозиторій:

https://github.com/Bitshkiper218/boardgame-server

Висновок:

Під час виконання практичної роботи №14 було успішно опановано базові принципи клієнт-серверної взаємодії за допомогою Node.js та фреймворку Express. Створено локальний веб-сервер, який успішно роздає статичні файли веб застосунку та надає власні REST API ендпоінти для предметної області «Каталогнастільних ігор» (варіант 4: /api/games та /api/games/:id). Клієнтський код перемкнуто на роботу з локальним сервером із збереженням патерну асинхронних запитів та обробки помилок (response.ok, try/catch). Проведено тестування кодів стану HTTP (200, 404), перевірено вкладку Network у DevTools та збережено фінальний код у репозиторії на GitHub.